WEEK 9 · บทที่ 8

Testbench Design & Functional Verification

การออกแบบ Testbench และการตรวจสอบเชิงฟังก์ชัน

self-checking testbench · $display / $monitor · $dumpvars & VCD · ModelSim-Altera · functional coverage

01 · RECAP

จาก Design สู่ Verification

6 สัปดาห์ที่ผ่านมาเราออกแบบวงจรได้หลายแบบ — แต่รู้ได้อย่างไรว่า "ถูกต้อง"?

W3-6

Combinational & Sequential

Half/Full Adder, Mux, Counter, Shift Register — ทดสอบด้วยตา (SW/LED) บน DE0

W7

Moore FSM

Traffic Light Controller — state diagram → Verilog

W8

Mealy FSM

Sequence Detector "1011" — overlapping detection

?

ปัญหา

วงจรซับซ้อนขึ้นเรื่อย ๆ — เสียบ DE0 แล้วดู LED ไม่พอที่จะจับบั๊กได้ทุกกรณีอีกต่อไป

01 · OVERVIEW

จุดประสงค์การเรียนรู้สัปดาห์นี้

1. เข้าใจโครงสร้าง testbench (DUT + stimulus + clock generator + checker) และเขียนได้ด้วยตนเอง
2. เขียน self-checking testbench ที่ตัดสิน PASS/FAIL อัตโนมัติด้วย $display แทนการไล่ดู waveform ด้วยตา
3. ใช้ $dumpfile/$dumpvars สร้างไฟล์ VCD และเปิดดู waveform ใน ModelSim-Altera เพื่อ debug
4. เข้าใจแนวคิด functional coverage — state coverage และ transition coverage สำหรับ FSM

01 · OVERVIEW

ทำไม Verification ถึงสำคัญ

50–70%

ของเวลาพัฒนาฮาร์ดแวร์ในอุตสาหกรรม ถูกใช้ไปกับการตรวจสอบ (verification) ไม่ใช่การออกแบบ (design) — ตัวเลขที่มักถูกอ้างอิงจากงานสำรวจอุตสาหกรรม (เช่น Wilson Research Group)

บั๊กที่หลุดไป FPGA จริง

debug บนฮาร์ดแวร์ยากกว่าบน simulation มาก — มองไม่เห็นค่าในทุก signal เหมือน waveform

FSM มี "มุมอับ" เยอะ

state/transition ที่ไม่เคยถูกกระตุ้นด้วย test case อาจซ่อนบั๊กไว้ได้นาน

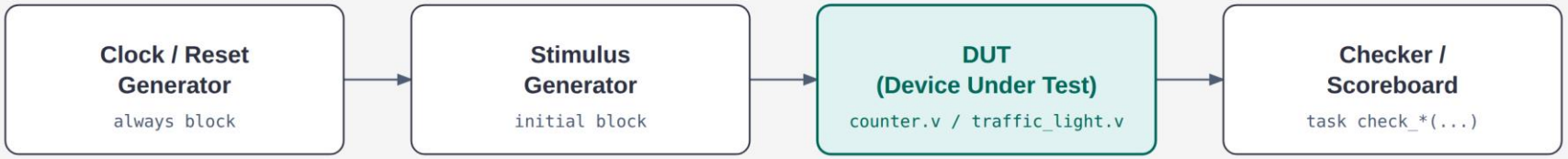
การนั่งดู LED ไม่ scale

ทดสอบ counter 10 ค่าด้วยตาไหว แต่ FSM ที่มีหลาย state และ timing จะนั่งไล่ดูไม่ไหว

02 · CONCEPT

โครงสร้าง Testbench

testbench คือ Verilog module ที่ไม่มี I/O จริง — มีหน้าที่ "ห่อหุ้ม" DUT แล้วป้อน/ตรวจสอบสัญญาณ



testbench ขับสัญญาณเข้า DUT ทางซ้าย แล้วตรวจผลลัพธ์ที่ออกทางขวา — เฉพาะกล่อง **DUT** เท่านั้นที่ถูก synthesize ลงฮาร์ดแวร์จริง

key idea: *testbench* ไม่ถูก synthesize ลงฮาร์ดแวร์จริง — ใช้ได้เฉพาะใน simulation เท่านั้น จึงเขียน syntax ที่ FPGA ทำไม่ได้ เช่น #delay, $display, `for` loop ไม่มีขอบเขต ได้อย่างอิสระ

03 · CODE

โครงร่าง Testbench ใน Verilog

module ไม่มี port — instantiate DUT ด้วย named mapping + สร้างสัญญาณนาฬิกาด้วย always block

VERILOG — tb_skeleton.v

```
module tb_counter;
  reg  clk = 0, rst, en, updown;
  wire [3:0] count;

  // instantiate DUT — named port mapping (ปลอดภัยกว่า positional)
  updown_counter DUT (
    .clk(clk), .rst(rst), .en(en), .updown(updown), .count(count)
  );

  // clock generator — คาบ 10 หน่วยเวลา (100 MHz หากหน่วยคือ ns)
  always #5 clk = ~clk;

  initial begin  // stimulus + checker ต่อจากนี้...
```

03 · CODE

Stimulus Generation & Timing Control

#delay ควบคุมเวลาใน simulation · @(posedge clk) รอจน edge ของนาฬิกาเกิดขึ้นจริง

VERILOG — apply stimulus

```
initial begin
  rst = 1; en = 0;
  @(posedge clk);   // รอ 1 edge
  rst = 0; en = 1;

  #20;              // รอ 20 หน่วยเวลา
  updown = 0;

  repeat (10) @(posedge clk);

  $finish;          // จบ simulation
end
```

#N

หน่วงเวลาแบบ absolute (N หน่วยเวลาตาม `timescale) — ใช้ในการจัด stimulus

@(posedge clk)

รอจนกว่า clock edge จะเกิดขึ้นจริง — สมจริงกว่า #delay เพราะซิงค์กับนาฬิกา

repeat(N) @(...)

รอ N ลูกของ clock — สะดวกเวลาต้องการ "ปล่อยให้วงจรนับไป N ครั้ง"

03 · CODE

Self-Checking: เทียบค่าคาดหวัง vs ค่าจริง

หัวใจของ self-checking testbench — โปรแกรมตัดสิน PASS/FAIL เอง ไม่ต้องอ่าน waveform ทีละเส้น

VERILOG — inline self-check

```
rst = 1; @(posedge clk); rst = 0; en = 1;
@(posedge clk); #1;    // #1 ให้ non-blocking update เสร็จก่อนเช็คค่า

if (count !== 4'd0) begin
  $display("[FAIL] after reset: expected=0 got=%0d @%0t", count, $time);
  errors = errors + 1;
end else begin
  $display("[PASS] after reset: count=%0d @%0t", count, $time);
end

// ใช้ !== / === (ไม่ใช่ != / ==) เพื่อจับค่า X/Z ด้วย
```

03 · CODE

ห่อ Checker เป็น Task

เขียนซ้ำทุกครั้งไม่สะดวก — ห่อ pattern เปรียบเทียบ+รายงานไว้ใน task เดียว เรียกใช้ซ้ำได้ทั้งไฟล์

VERILOG — reusable checker task

```
integer errors = 0;

task check_count(input [3:0] exp, input [127:0] label);
  begin
    if (count !== exp) begin
      $display("[FAIL] %s: expected=%0d got=%0d @%0t", label, exp, count, $time);
      errors = errors + 1;
    end else $display("[PASS] %s: count=%0d", label, count);
  end
endtask

// เรียกใช้: check_count(4'd0, "after reset");
```

03 · CODE

เรียก check_count() จริง — ดูผลทีละครั้ง

3 จังหวะเวลาในการทดสอบ mod-10 counter เดียวกัน — Transcript จะพิมพ์ผลลัพธ์ทีละบรรทัดตามลำดับนี้

เวลาที่ 1

```
check_count(4'd0, "after reset");
```

```
[PASS] after reset: count=0
```

PASS

เวลาที่ 2

```
check_count(4'd0, "wrap 9->0 after 10 counts");
```

```
[FAIL] wrap 9->0 after 10 counts: expected=0 got=1 @115
```

FAIL

เวลาที่ 3

```
check_count(4'd9, "wrap 0->9 counting down");
```

```
[PASS] wrap 0->9 counting down: count=9
```

PASS

errors จะถูกบวกเพิ่มเฉพาะบรรทัดที่ [FAIL] เท่านั้น — ท้ายไฟล์จึงเช็ค if (errors==0) เพื่อสรุปผลรวมทั้งไฟล์ได้ในบรรทัดเดียว

03 · CODE

$display vs $monitor vs $strobe

system task สามตัวที่ใช้บ่อยที่สุดใน testbench — ต่างกันที่ "เวลา" ที่พิมพ์ค่า

Task	พิมพ์เมื่อไหร่	เหมาะกับ
$display(...)	ทันทีที่บรรทัดถูก execute (ครั้งเดียว)	checkpoint message, debug snapshot ณ จุดใดจุดหนึ่ง
$monitor(...)	อัตโนมัติทุกครั้งที่ argument เปลี่ยนค่า (สูงสุด 1 ครั้ง/time step)	ติดตามสัญญาณต่อเนื่องตลอด simulation โดยไม่ต้องเรียกซ้ำ
$strobe(...)	ท้ายสุดของ time step — หลัง non-blocking update ทุกตัว settle แล้ว	ดูค่าที่ "นิ่ง" จริงในรอบนั้น หลีกเลี่ยงปัญหา race condition

03 · CODE

ตัวอย่างจริง: 3 task พิมพ์ค่าเดียวกัน คนละเวลากัน

เกิด race condition แบบเดียวกับ non-blocking assignment — สังเกตค่า count ที่แต่ละ task "เห็น" ตอน t=10 ไม่เท่ากัน

VERILOG

```
reg [3:0] count = 0;
always @(posedge clk) count <= count + 1;

initial $monitor("[MONITOR] count=%0d @%0t",
count, $time);

always @(posedge clk) begin
  $display("[DISPLAY] count=%0d @%0t", count,
$time);
  $strobe ("[STROBE]  count=%0d @%0t", count,
$time);
end

// ทั้งสาม always/initial trigger ที่ posedge clk
// เวลาเดียวกัน (t=10) แต่คนละ "region" ของ event
```

[DISPLAY]

Active region — ทำงานทันทีตอน edge ก่อน count <= ... จะ settle

[DISPLAY] count=0 @10

[MONITOR]

Inactive region — พิมพ์อัตโนมัติทันทีที่เห็นค่าเปลี่ยน (หลัง update)

[MONITOR] count=1 @10

[STROBE]

Postponed region — รับประกันว่าทุกค่า non-blocking settle แล้ว

[STROBE] count=1 @10

03 · CODE

Waveform Dumping: $dumpfile / $dumpvars

เมื่อ self-check ยัง FAIL อยู่ waveform คือเครื่องมือถัดไปสำหรับ debug เชิงลึก

VERILOG

```
initial begin
  $dumpfile("tb_counter.vcd");
  $dumpvars(0, tb_counter);
  ...
end
```

$dumpfile(name)

กำหนดชื่อไฟล์ .vcd (Value Change Dump) ที่จะบันทึกทุกการเปลี่ยนค่าสัญญาณ

$dumpvars(0, mod)

อาร์กิวเมนต์ 0 = ลงลึกทุกระดับ (รวม signal ภายใน DUT), mod = scope ที่จะเริ่มจับ

เปิดใน ModelSim

vsim โหลด .vcd แล้ว add wave สัญญาณที่สนใจ เพื่อดูรูปคลื่นจริงประกอบผล PASS/FAIL

ModelSim-Altera Workflow

ขั้นตอนมาตรฐานตั้งแต่ compile จนถึงดู waveform

1. vlib work → สร้าง simulation library ชื่อ "work" (ทำครั้งเดียวต่อโปรเจกต์)
2. vlog counter.v tb_counter.v → compile ไฟล์ RTL และ testbench เข้า library
3. vsim tb_counter → โหลด testbench เข้า simulation kernel (เลือก top module)
4. add wave -radix binary /tb_counter/* → เพิ่มสัญญาณทั้งหมดเข้า Wave window (หรือเลือกเฉพาะตัว)
5. run -all → รันจนกว่า $finish จะถูกเรียก แล้วอ่านผล PASS/FAIL จาก Transcript

Testbench Syntax — กฎพื้นฐานที่ต้องรู้

สรุปไวยากรณ์หลักที่ใช้ซ้ำในทุก testbench ตลอดบทนี้

1. `timescale 1ns/1ps` → กำหนดหน่วยเวลา/ความละเอียดที่ #delay ทุกตัวอ้างอิง (บรรทัดแรกของไฟล์)
2. module tb_name; ... endmodule → ไม่มี port list เพราะ testbench ไม่ถูก synthesize ลงฮาร์ดแวร์จริง
3. reg สำหรับสัญญาณที่ testbench เป็นผู้ขับค่า (stimulus/clock) / wire สำหรับสัญญาณที่รับจาก DUT
4. initial begin...end → รันครั้งเดียวที่ time 0 ตั้งค่าเริ่มต้น + เขียนลำดับ stimulus หลัก
5. always (ไม่มี sensitivity list + #delay) → สร้าง clock วนซ้ำ; #N / @(posedge sig) ควบคุมจังหวะเวลา

05 · SYNTAX

5 กฎนี้ในโค้ดจริง — พร้อมผลลัพธ์ทีละขั้น

โครงร่างเดียวกับ tb_updown_counter.v — ตัวเลข (1)-(5) ชี้กลับไปยังกฎในสไลด์ก่อนหน้า

VERILOG — tb_updown_counter.v (skeleton)

```
`timescale 1ns/1ps            // (1)
module tb_updown_counter;     // (2)
  reg clk=0, rst, en, updown; // (3)
  wire [3:0] count;           // (3)

  updown_counter dut(.clk(clk), .rst(rst),
    .en(en), .updown(updown), .count(count));

  always #5 clk = ~clk;         // (5)

  initial begin               // (4)
    rst=1; en=0; updown=1;
    @(posedge clk); #1; rst=0;   // (5)
    check_count(4'd0, "after reset");
  end
endmodule
```

0 ns

`timescale` มีผลแล้ว — ทุก #N จากนี้นับเป็น N × 1ns; initial block เริ่มทำงาน: clk=0, rst=1, en=0, updown=1

5 ns

always block ครบ #5 ครั้งแรก → clk พลิกเป็น 1 (นี่คือ posedge แรกที่ initial block รออยู่)

5 ns (+1)

@(posedge clk) หลุดบล็อก แล้ว #1 หน่วงให้ rst=0 มีผลที่ 6ns ไม่ชนกับขอบ clock พอดี

6 ns

rst=0 มีผลจริง — เข้าเงื่อนไข check_count() ครั้งแรกด้วยค่า count ที่ reset แล้ว

10 ns, 15 ns, …

always #5 วน clk สลับ 0/1 ต่อเนื่องทุก 5ns — เป็นคาบ clock 10ns ไปตลอดการจำลอง

System Tasks อ้างอิงเพิ่มเติม

ต่อจาก $display/$monitor/$strobe และ $dumpfile/$dumpvars ที่เจอไปแล้ว

System Task	หน้าที่	ตัวอย่าง
$finish	จบ simulation ทันที (kernel ออกจากการรัน)	$finish;
$stop	หยุดชั่วคราว กลับสู่ prompt แบบ interactive — debug ต่อได้โดยไม่ต้อง restart	$stop;
$time / $realtime	คืนค่าเวลาปัจจุบันของ simulation แบบ integer / real	$display("@%0t", $time);
$random	สุ่มค่า integer 32-bit — ใช้สร้าง stimulus แบบสุ่มสำหรับ stress test	data = $random;

05 · SYNTAX

Task vs Function ใน Testbench

สองวิธีห่อโค้ดให้เรียกใช้ซ้ำได้ — เลือกผิดแบบ compile ไม่ผ่าน

	task	function
ค่าที่คืนกลับ	ไม่มี (ใช้ output/inout argument แทนได้)	ต้องคืนค่าเดียวเสมอ ผ่านชื่อ function เอง
Timing control (#, @, wait)	ใช้ได้ — เหมาะกับ stimulus/checker ที่ต้องรอเวลา	ห้ามใช้ — ต้องทำงานเสร็จภายใน 0 หน่วยเวลา
เรียกตัวอื่นต่อ	เรียกได้ทั้ง task และ function อื่น	เรียกได้เฉพาะ function อื่น (เรียก task ไม่ได้)
ตัวอย่างที่ใช้ในบทนี้	check_count(...), pulse_tick	คำนวณค่า combinational เช่น expected = calc(...)

06 · SIM LEVELS

รัน Testbench ในแต่ละขั้นของ Design Flow

testbench ตัวเดิม ใช้ซ้ำได้ทั้ง 3 จุดตรวจสอบ — เปลี่ยนแค่ "netlist" ที่ป้อนเข้าไปจำลอง

① RTL Design (เขียน Verilog) → RTL/Behavioral Simulation กับซอร์ส .v ตรง ๆ ไม่มี delay จริง (เร็วที่สุด ใช้ตลอดบทนี้)

② Synthesis (Analysis & Synthesis) → gate-level netlist → Post-Synthesis Simulation: รันซิมเดิมกับ netlist (ยัง zero-delay) ยืนยันฟังก์ชันไม่เปลี่ยน

③ Place & Route (Fitter) + TimeQuest สร้าง SDF (delay จริง) → Post-Implementation (Timing) Simulation ตรวจ setup/hold และ glitch

④ Program FPGA (DE0) → ทดสอบบนฮาร์ดแวร์จริง — ด่านสุดท้ายหลังผ่านทั้ง 3 ระดับ simulation แล้ว

RTL vs Post-Synthesis vs Post-Implementation

netlist และ delay model ที่ต่างกัน ทำให้แต่ละระดับจับบั๊กคนละประเภท

ระดับ Simulation	Netlist / Delay Model	ตรวจพบบั๊กประเภทไหน
RTL / Behavioral (ก่อน synthesis)	ซอร์ส Verilog ตรง ๆ — zero-delay ตามที่เขียน #delay เอง	ตรรกะ/อัลกอริทึมผิด, edge case ไม่ครบ, coding error ทั่วไป
Post-Synthesis (Functional)	Gate-level netlist หลัง Analysis & Synthesis — ยังไม่มี delay จริง	synthesis ตีความโค้ดผิดจากที่ตั้งใจ เช่น เผลอสร้าง latch หรือ optimize logic ที่ไม่ควรหาย
Post-Implementation (Timing)	Gate-level netlist หลัง Fitter + SDF back-annotation จาก TimeQuest	glitch/hazard บนสัญญาณ combinational, setup/hold violation, ไม่ผ่าน Fmax เป้าหมาย

06 · SIM LEVELS

พฤติกรรมที่เปลี่ยนไปหลัง Synthesis

สิ่งที่ RTL simulation มองไม่เห็น แต่ post-synthesis netlist เจอ

!

Latch ที่ไม่ตั้งใจ

if ไม่มี else หรือ case ไม่มี default → RTL sim ดูปกติ เพราะ simulator จำค่าเก่าไว้ แต่ netlist จริงมี latch ที่ glitch ได้ง่าย

!

สถานะตอนเปิดเครื่อง

RTL testbench มักตั้งค่าเริ่มต้นให้ reg ชัดเจน แต่ flip-flop จริงบนชิปอาจ power-up เป็นค่าไม่แน่นอนถ้า reset ไม่ครอบคลุมทุก state

i

Logic ที่ถูกตัดทิ้ง

โค้ดที่เขียนไว้แต่ไม่ส่งผลต่อ output จริง (dead logic) จะถูก optimize หายไปจาก netlist — signal ภายในนั้นอาจ observe ไม่ได้อีกต่อไป

✓

ทำไมยังต้องรันซิมนี้

เพื่อยืนยันว่าพฤติกรรมที่สังเกตได้จากภายนอกของ netlist ยังตรงกับ RTL แม้โครงสร้างภายในจะเปลี่ยนไปตาม optimization

06 · SIM LEVELS

พฤติกรรมที่เปลี่ยนไปหลัง Implementation (Timing)

ด่านสุดท้ายก่อนเชื่อว่าโค้ดที่ผ่านทุก simulation จะทำงานได้จริงบน DE0

!

Glitch / Hazard

RTL sim แบบ zero-delay ไม่เคยเห็น glitch แต่วงจรจริงมี delay ของแต่ละเกต ทำให้ combinational output แกว่งสั้น ๆก่อนนิ่ง — post-fit timing sim จับสิ่งนี้ได้

!

Setup / Hold Violation

ถ้าเส้นทาง combinational ยาวเกินไปเทียบกับคาบสัญญาณนาฬิกา ค่าจะมาไม่ทันขอบ clock ถัดไป — TimeQuest และ post-fit sim รายงานปัญหานี้

↺

Timing Closure

ถ้า design ไม่ผ่าน timing ต้องย้อนกลับไปแก้ RTL (เช่น เพิ่ม pipeline stage) แล้ววนซิม-synthesize-fit ใหม่จนผ่าน Fmax เป้าหมาย

✓

ก่อน Program ลง DE0

Post-implementation timing simulation คือด่านสุดท้ายก่อนเชื่อว่าวงจรบนฮาร์ดแวร์จริงจะทำงานถูกต้องตามความเร็วที่ต้องการ

Self-Checking TB: Mod-10 Up/Down Counter

DUT จากสัปดาห์ที่ 6 — วางแผนทดสอบก่อนลงมือเขียนโค้ดเสมอ

กรณีทดสอบ	การกระตุ้น (stimulus)	ค่าที่คาดหวัง
Reset	rst = 1 หนึ่ง clock edge	count = 0
นับขึ้นปกติ	en=1, updown=1 ให้ครบ 10 clock	count วนกลับ 9 → 0 (mod-10)
นับลงปกติ	en=1, updown=0 หนึ่ง clock จาก 0	count wrap 0 → 9
Enable = 0	en = 0 ระหว่างนับ	count ต้องค้างค่าเดิม ไม่เปลี่ยน

Counter Testbench — โค้ดตัวอย่าง

VERILOG — tb_updown_counter.v (ย่อ)

```
reg clk=0, rst, en, updown;  integer errors=0;
always #5 clk = ~clk;

initial begin
  $dumpfile("tb_counter.vcd"); $dumpvars(0, tb_updown_counter);
  rst=1; en=0; updown=1; @(posedge clk); #1;
  check_count(4'd0, "after reset");
  rst=0; en=1; repeat (10) @(posedge clk); #1;
  check_count(4'd0, "wrap 9->0 after 10 counts");
  updown=0; @(posedge clk); #1;
  check_count(4'd9, "wrap 0->9 counting down");
  if (errors==0) $display("TESTBENCH PASSED");
  else $display("FAILED: %0d errors", errors);
  $finish;
end
```

ทำไม Verify FSM ยากกว่า Counter

counter มีค่าให้เช็คเป็นเส้นตรง แต่ FSM มีทั้ง "state" และ "transition" ที่ต้องครอบคลุมให้ครบ

①

State Coverage

ต้องมั่นใจว่า stimulus พาวงจรผ่านทุก state อย่างน้อยคนละ 1 ครั้ง (RED, GREEN, YELLOW)

②

Transition Coverage

ทุกลูกศรใน state diagram ต้องถูกกระตุ้นอย่างน้อย 1 ครั้ง รวมถึง self-loop และเส้นทาง reset

③

Timing ต้องถูกด้วย

ไม่ใช่แค่ state ถูก แต่ "ระยะเวลา" ที่อยู่ในแต่ละ state ต้องตรง spec (เช่น RED ต้องอยู่ 4 ติ๊ก ไม่ใช่ 3 หรือ 5)

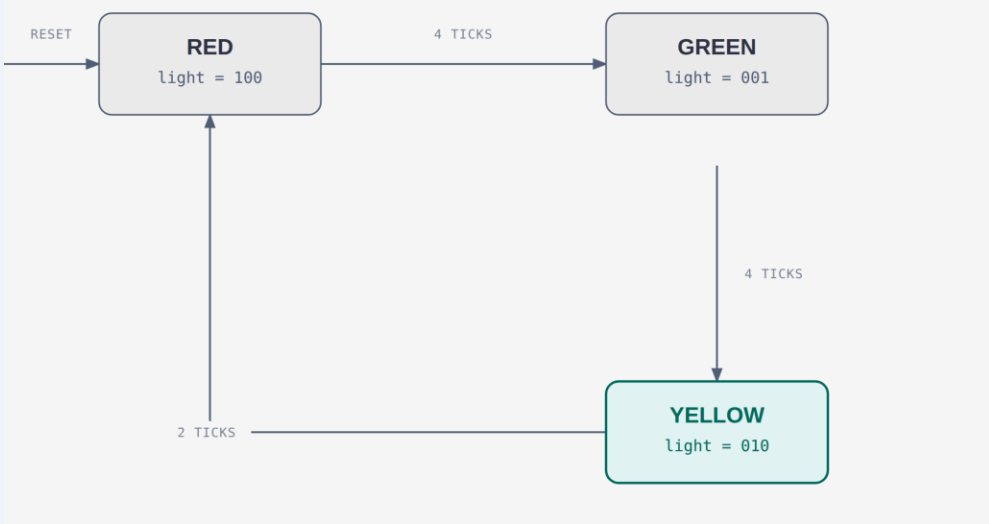
④

เทคนิค: แยก tick ออกมา

จำลอง 4 วินาทีจริงที่ 50 MHz ใช้ clock cycle เยอะเกินไป — ป้อน "tick" ตรงจาก testbench แทนเพื่อซิมเร็วขึ้น

Traffic Light Controller — State Diagram

DUT จากสัปดาห์ที่ 7 — Moore FSM สามสถานะ ก่อนลงรายละเอียดตารางและโค้ด testbench



Moore machine

output (light) ขึ้นกับ state เท่านั้น ไม่ขึ้นกับ input โดยตรง — ต่างจาก Mealy

ต้องตรวจทั้ง state และ transition

แค่ค่า light ถูกยังไม่พอ ต้องนับจำนวน tick ที่อยู่ในแต่ละ state ให้ตรง spec ด้วย

YELLOW คือจุดที่มักคำนวณผิด

อยู่แค่ 2 ticks (สั้นกว่า RED/GREEN ที่ 4 ticks) — นักศึกษามักลืมปรับเงื่อนไข count

Self-Checking TB: Traffic Light FSM (Moore)

DUT จากสัปดาห์ที่ 7 — ตารางลำดับผลลัพธ์ที่คาดหวัง คือพิมพ์เขียวของ checker

State	light[2:0] = {R,Y,G}	จำนวน tick ที่ต้องอยู่	การเปลี่ยนถัดไป
หลัง reset	100 (RED)	—	→ RED
RED	100	4 ticks	→ GREEN
GREEN	001	4 ticks	→ YELLOW
YELLOW	010	2 ticks	→ RED (วนกลับ)

Traffic Light Testbench — โค้ดตัวอย่าง

VERILOG — tb_traffic_light.v (ย่อ)

```
reg clk=0, rst, tick;  wire [2:0] light;  integer errors=0;
always #5 clk = ~clk;

task pulse_tick; begin tick=1; @(posedge clk); tick=0; end endtask

initial begin
  rst=1; tick=0; @(posedge clk); rst=0; #1;
  check_light(3'b100, "reset -> RED");

  repeat (4) pulse_tick; #1; check_light(3'b001, "RED(4)->GREEN");
  repeat (4) pulse_tick; #1; check_light(3'b010, "GREEN(4)->YELLOW");
  repeat (2) pulse_tick; #1; check_light(3'b100, "YELLOW(2)->RED (loop)");

  if (errors==0) $display("TESTBENCH PASSED"); $finish;
end
```

แนวคิด Functional Coverage

"code coverage" บอกว่าโค้ดถูกรันครบไหม — "functional coverage" บอกว่า สถานการณ์ ที่ spec ต้องการถูกทดสอบครบไหม

ประเภท Coverage	ตอบคำถามว่า...	วิธีวัดแบบง่าย (สัปดาห์นี้)
Code Coverage	ทุกบรรทัด/branch ของโค้ด DUT ถูก execute หรือยัง?	เครื่องมือ coverage ใน ModelSim (ขั้นสูง — ยังไม่ใช้สัปดาห์นี้)
State Coverage	stimulus พาวงจรผ่านครบทุก state หรือยัง?	ตัวแปร integer นับจำนวนครั้งที่แต่ละ state ถูกเข้า แล้ว $display สรุปท้าย test
Transition Coverage	ทุกเส้นทางใน state diagram ถูกกระตุ้นหรือยัง?	ออกแบบ test case ให้ครอบคลุมทุกลูกศรตามที่ร่าง state diagram ไว้ตั้งแต่ต้น

SystemVerilog covergroup/coverpoint คือเครื่องมือมาตรฐานอุตสาหกรรมสำหรับงานนี้ — เกินขอบเขตคอร์ส แต่ควรรู้จักชื่อไว้

Debug Checklist: บั๊กที่พบบ่อยใน Testbench

1. ใช้ == / != แทน === / !== → ตรวจไม่เจอค่า X/Z ที่หลุดมาจาก DUT ที่ไม่ได้ reset ครบ
2. เช็คค่าทันทีที่ posedge clk โดยไม่หน่วง #1 → อ่านค่าเก่าก่อน non-blocking (<=) จะ update เสร็จ (race condition)
3. ผสม blocking (=) กับ non-blocking (<=) ปนกันใน always block เดียวกันของ DUT/testbench
4. ลืม deassert reset ก่อนเริ่ม stimulus จริง → วงจรค้างอยู่ที่สถานะ reset ตลอดการทดสอบ
5. ลืมเรียก $finish → simulation ไม่จบเอง (ต้องรอ timeout หรือ kill process เอง)

12 · BEST PRACTICE

แนวปฏิบัติที่ดีในการเขียน Testbench

01

แยกส่วนให้ชัด

stimulus, checker, DUT instantiation แยกเป็นคนละ block/task — อ่านและแก้ไขง่าย

02

Self-checking เสมอ

ทุก testbench ต้องจบด้วย PASS/FAILED ชัดเจน ไม่พึ่งการนั่งไล่ waveform เป็นวิธีหลัก

03

Parameterize timing

ใช้ parameter สำหรับ clock period/จำนวน tick เพื่อปรับความเร็ว simulation ได้ง่าย

04

นับ error เสมอ

ตัวแปร errors สะสมทุก mismatch แล้วสรุปผลรวมบรรทัดสุดท้ายของ simulation

Gemini ช่วยอะไรได้บ้างสัปดาห์นี้

1. "ช่วยสร้าง self-checking testbench สำหรับโมดูล [วางโค้ด DUT] พร้อม task checker และนับ error"
2. "testbench นี้รายงาน [FAIL] ที่บรรทัด X ทำไม assertion ถึงไม่ผ่าน ช่วย debug ให้หน่อย" (แนบโค้ด + log)
3. "อธิบายความแตกต่างระหว่าง code coverage กับ functional coverage ให้เข้าใจง่ายด้วยตัวอย่าง FSM"
4. "ช่วยออกแบบชุด test case ให้ครอบคลุมทุก state และทุก transition ของ FSM นี้ [วาง state diagram/โค้ด]"

สรุปบทที่ 8 — Testbench & Functional Verification

- โครงสร้าง testbench: clock/reset generator + stimulus + DUT + checker ✓
- Self-checking ด้วย $display/$monitor/$strobe และ task checker ที่นับ error ✓
- Waveform dump ด้วย $dumpfile/$dumpvars → เปิดดูใน ModelSim-Altera ✓
- รัน testbench เดิมได้ทั้ง 3 ระดับ: RTL → Post-Synthesis → Post-Implementation (Timing) ✓
- State coverage และ transition coverage สำหรับ FSM verification ✓
- Lab: self-checking TB สำหรับ Mod-10 Counter (W6) และ Traffic Light FSM (W7) ✓

▶ สัปดาห์ที่ 10: ALU & RISC-V Single Cycle Pipeline — เริ่มสร้าง RV32I ALU + Register File + Control Unit